

Chapter 1

Modularizing Embedded Projects

ENEL712 Embedded Systems Design — Lecture 10

Summary

Modular design principles in embedded C: how to split a project into modules, source files, and functions; driver levels; encapsulation in C; and a 6-step modular design recipe.

Topics Covered

- Core concepts of modularization
- Structuring modular software
- Advantages and disadvantages
- Specialized modular techniques
- Six-step modular design recipe
- Encapsulation and scoping in C
- Performance implications

1.1 Core Concepts of Modularization

- Modularization splits a large program into manageable components: source files, classes, and functions.
- Group common functionality into modules with focused requirements.
- Modules can be unique or interchangeable; modular programming is a key part of architectural design.
- Applies across software levels (drivers to apps) and even hardware.

1.2 Structuring Modular Software

Level	Description
Modules	High-level abstract groupings of functionality (e.g., Communications, Controller, UI). Typically 3–5.
Source files	Each module = one or more source files (one per module is often easiest).
Functions	Each source file = one or more focused functions (recommended 3–6).

1.3 Advantages and Disadvantages

Advantages	Disadvantages
Promotes top-down architecture and good design	More files create clutter
Easier to develop, debug, test, and maintain	Finding functionality across many files can be hard
Enables isolation for testing	Header files needed to link functions across files
Provides basic encapsulation by controlling access	Function call overhead (push/pop on stack)
Enables code reuse across projects	—

1.4 Specialized Modular Techniques

1.4.1 Driver levels

Level	Role
Low-level	Operates at the register level (most hardware-dependent).
Mid-level	Basic operations like sending/receiving individual bytes.
High-level	Specific complex functionality calling mid-level functions.

1.4.2 Wrapper functions

Provide a simpler calling interface for complex functions, often setting default values or converting argument formats without changing core behaviour.

1.5 Six-Step Modular Design Recipe

- Module identification: divide specs into 3–5 main groups (e.g., physical devices, user I/O, algorithms, data).
- Source file identification: usually one per module unless complexity demands more.
- Function identification: split file requirements into focused functions; avoid duplication; use wrappers.

- Data identification: decide how data is passed (consider structures and pointers for many variables).
- Interrupt identification: pick events for ISRs based on timing or risk of missing data.
- Code reuse: write modules (e.g., SPI) to be platform/application-independent where possible.

1.6 Encapsulation and Scoping in C

- Header (.h): public API — declarations of functions and types other modules use; extern declares variables/functions defined elsewhere.
- Source (.c): implementation; static gives functions/variables internal linkage and hides them from other modules.
- Layered driver approach simplifies application code (e.g., readSPIReg(device, address) hides chip select and register details).

1.7 Performance Implications

Function calls require pushing parameters/return address/registers onto the stack and popping them on return. Lots of jumping = lots of push/pop. For short, time-critical code this overhead can matter.

1.8 Use Cases & Applications

- Driver reuse: an SPI module written against a generic 'send byte / receive byte' interface drops into the next project on a different MCU with only the lowest layer rewritten.
- Team scaling: separate source files let two engineers work on Comms and UI independently with minimal merge conflicts.
- Unit testing: modules with narrow header APIs can be linked against a PC build harness and exercised without hardware.

1.9 Worked Example: Layered SD-Card Driver

Layer	Functions	What it knows about
Low (mcu_spi.c)	spi_init(), spi_xfer(byte)	AVR SPI registers
Mid (sd_block.c)	sd_read_block(lba, *buf), sd_write_block(lba, *buf)	SD command set, CRCs
High (fs_fat.c)	fopen(), fread(), fwrite()	File system semantics
App (logger.c)	log_event(struct e)	Business logic only

Swapping the MCU only changes mcu_spi.c — the SD command logic, FAT layer, and application stay identical.

1.10 Code Snippets

1.10.1 Header file (public API)

```
// uart.h
#ifndef UART_H
#define UART_H

#include <stdint.h>

void uart_init(uint32_t baud);
void uart_putc(uint8_t c);
uint8_t uart_getc(void);

#endif
```

1.10.2 Source file with private statics

```
// uart.c
#include "uart.h"
#include <avr/io.h>

static volatile uint8_t rx_buf[64]; // private to this file
static volatile uint8_t rx_head, rx_tail;

void uart_init(uint32_t baud) { /* ... */ }
void uart_putc(uint8_t c)      { /* ... */ }
uint8_t uart_getc(void)        { /* ... */ }
```

1.10.3 Wrapper function example

```
// Convenience wrapper that hides the SPI mode and chip-select details.
uint8_t adc_read_channel(uint8_t ch) {
    uint8_t cmd[3] = { 0x06 | (ch >> 2), ch << 6, 0x00 };
    spi_select(ADC_CS);
    spi_xfer_buf(cmd, 3);
    spi_deselect(ADC_CS);
    return ((cmd[1] & 0x0F) << 8) | cmd[2];
}
```

Key Definitions

Module

High-level abstract grouping of related functionality (e.g., UI, Comms, Controller).

Source File

Concrete .c file implementing a module's functions and private data.

Wrapper Function

Function simplifying the calling interface for a more complex underlying function.

Low-Level Driver

Code operating directly on hardware registers.

High-Level Driver

Code performing complex device functionality via mid-/low-level calls.

Encapsulation (in C)

Hiding internals using headers for the public API and static for private members.

Function Call Overhead

Cost of pushing/popping context on the stack at every call/return.

Top-Down Design

Plan architecture (modules, files, functions) before implementation.

ISR

Interrupt Service Routine; runs in response to a hardware signal.

Translation Unit

A .c source file plus everything it #includes after preprocessing — the unit the compiler turns into one .o.

API (Application Programming Interface)

Set of declarations a module exposes for callers to use; in C, this lives in the header file.

Volatile Keyword

Tells the compiler not to optimise away accesses to a variable; required for hardware registers and ISR-shared state.

Sources

- Lecture 10.pdf